Secure Exit

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

BAGYA VENKATESH
192324263RA

CO3-AT1-Assignment

View Only

← Back

QUESTIONS

Q1 ✓
Banking Transaction System – Exception Handling
10 marks

Q2 ✓
Hospital Patient Registration – User-Defined Exception
10 marks

Q3 ✓
E-Commerce Order Processing – Built-in Exceptions
10 marks

Q4 ✓
Online Food Delivery – Multiple Threads and Execution Order
10 marks

Q5 ✓
Manufacturing Plant – Producer-Consumer Problem
10 marks

5/5 answered

Q5 Manufacturing Plant – Producer-Consumer Problem

10 marks

A manufacturing plant has one production thread and one packaging thread. A shared buffer can contain only one product.

Implement the producer-consumer solution using synchronized, wait(), and notify().

The program must:
• Make the producer wait when the buffer is full.
• Make the consumer wait when the buffer is empty.
• Prevent duplicate consumption.
• Ensure that products are consumed only after production.
• Demonstrate correct coordination between the two threads.

Analyze what incorrect behavior could occur if synchronization and wait/notify are removed.
Sample Test Cases:
Input / Scenario Expected Result Analysis Focus
Products = P1, P2, P3 P1 → P2 → P3 are produced and consumed once each Normal execution
Consumer starts first Consumer waits until a product is available Concurrency / design

Buffer capacity = 1; Products = P1, P2 Producer waits while P1 remains unconsumed Exception / boundary

Your Code

```
class Plant{
    String p;
    boolean full =false;
    synchronized void produce(String x)thr
        while(full) wait();
        p=x;
        full=true;
        System.out.println("Produced: "+p)
        notify();
    }
    synchronized void consume() throws Exc
        while(!full)wait();
        System.out.println("Consumed: "+p)
        full=false;
        notify();

    }
    public static void main(String[] args)
        Plant b = new Plant();
        Thread producer = new Thread(()->
            try{
                b.produce("p1");
                b.produce("p2");
                b.produce("p3");
            }catch(Exception e){}
        });
        Thread consumer = new Thread(()->
            try{
                b.consume();
                b.consume();
                b.consume();
            }catch(Exception e){}
        });
        producer.start();
        consumer.start();
    }
}
```

Input

stdin input

Output

```
Produced: p1
Consumed: p1
Produced: p2
Consumed: p2
Produced: p3
Consumed: p3
```

Submitted